API REST (Definiciones)

- API REST (Definiciones)
  - Recursos
  - Identificador de Recurso
    - Identificadores en plural
    - Identificadores en Singular
    - Acción/Verbo no debe ser parte del Identificador(url)
  - Endpoint
  - Recurso Vs Identificador de Recurso Vs Endpoint
  - Acciones/Verbos: Qué hacer con el recurso?
  - Metadatos
  - Representaciones
    - El cliente puede indicar que representación quiere del recurso
    - Mediante encabezados de negociación de contenido
    - Mediante extensiones de archivo
  - Transferencia del Estado Representacional
    - Estado Representacional
    - Transferencia

En REST transferimos el estado de cosas (recursos) mediante diferentes tipos de representaciones
REST se centra en estas cosas/recursos. Estos recursos tienen estados que van a ser transferidos.

Recursos

Cualquier cosa puede ser un recurso, una imagen, una persona, un reporte.
Estos recursos son lo que definen el servicio que nuestra API va a proporcionar.
Un recurso va tener:

- **Identificador**: La URL del recurso.
- **Acciones**: Qué se puede hacer con el recurso.
- **Metadatos**: Tamaño, cuando fue modificado, tipo de contenido, etc.
- **Representaciones** Las formas de representar el recurso (Xml, Json. Binario)
- **Estado representacional** Se refiere al estado de un recurso mediante una representación

Una API REST transfiere el estado de un recurso por medio de representaciones de el.

API REST (Definiciones)

Identificador de Recurso

Un identificador de recurso en el contexto de REST se refiere a la URL para acceder a el.
Los clientes acceden al recurso mediante esta URL, el patrón que debe seguir esta URL debe ser predecible, uniforme y jerárquico, por ejemplo si el recurso fuera un catálogo de usuarios:

Método Http	Identificador	URI	Acción
`GET`	`/usuarios`	`https://dominio.com/api/usuarios`	Regresa todos los usuarios
`GET`	`/usuarios/34`	`https://dominio.com/api/usuarios/34`	Regresa el usuario con el ID 34
`POST`	`/usuarios`	`https://dominio.com/api/usuarios`	Agregará un nuevo usuario
`PUT`	`/usuarios/34`	`https://dominio.com/api/usuarios/34`	Modifica el usuario 34
`DELETE`	`/usuarios/34`	`https://dominio.com/api/usuarios/34`	Eliminar el usuario 34

Los nombres de los recursos:

- **Deben ser sustantivos** claros y descriptivos.
- **Deben estar en minúsculas** ya que las URL son case sensitive.
- **Deben usar guión medio** como separador (en caso de necesitarlo) en lugar de guión bajo o Camel Case.

Correcto	Incorrecto
`GET /ordenes-cliente/3544BC`	`GET /ordenesCliente/3544BC`
`GET /clientes/`	`GET /ordenes_cliente/3544BC`
`GET /productos/?id=123`	`GET /PRODUCTOS/?id=123`

API REST (Definiciones)

Identificadores en plural

Los nombres de los recursos deben estar en plural, no es obligatorio pero si una convención muy extendida, que siguen todas las APIS Web, de manera que usaremos:

Identificador	Para:
`/usuarios/`	Obtener toda la lista de usuarios
`/usuarios/1`	Obtener el usuario con el `id=1`
`/usuarios/?id=1`	Obtener el usuario con el `id=1`
`/articulos/nombreArticulo`	Primero el nombre del recurso en plural Después el recurso en específico en singular

Identificadores en Singular

Se pueden incluir identificadores de recursos en singular pero solo si el recurso es único:

Identificador	Para:
`GET /api/status`	Obtener el estatus del servicio
`GET /api/config`	Obtener la configuración
`GET /api/version`	Obtener la version
`GET /api/login`	Logearse

Acción/Verbo no debe ser parte del Identificador(url)

No se debe de incluir la acción en el identificador del recurso. En Rest la acción la define em método de la petición Http:

- `https://dominio.com/api/ConsultaUsuarios` (INCORRECTO / MALA PRACTICA)
- `https://dominio.com/api/Borrarusuario/1` (INCORRECTO / MALA PRACTICA)

API REST (Definiciones)

Endpoint

El término endpoint se refiere al punto de contacto entre cliente y servidor.
Es la URL a donde el cliente envía una petición Http con un método para acceder a un recurso.

Recurso Vs Identificador de Recurso Vs Endpoint

Recurso	Identificador de Recurso	Endpoint
Cualquier cosa que la API exponga	Parte de la URL que identifica un solo recurso o instancia en una colección de recursos	La URL, el dominio y la ruta a la que se envía la petición (Método) Puede contener un identificador de recurso>
Clientes	`/clientes/2`	`https:/dominio.com/clientes/2`

API REST (Definiciones)

Acciones/Verbos: Qué hacer con el recurso?

REST se basa en el protocolo HTTP y este protocolo define una serie de verbos o acciones (también llamados métodos).
Estos métodos pueden usarse para definir que acción vamos a realizar sobre el recurso.
LAs acciones dependen del tipo de API y de el servicio que se desee dar, pero hay ciertas acciones
que son comunes en todas las APIs:

Método HTTP	Acción a realizar
`GET`	Obtiene recurso del servidor
`POST`	Envía recurso al servidor para darlo de alta
`PUT`	Actualiza todo un recurso completamente
`PATCH`	Actualiza solamente partes de un recurso
`DELETE`	Elimina recurso del servidor
`HEAD`	Lee un recurso sin retornarlo, obteniendo solo los encabezados
`OPTIONS`	Obtiene los métodos de un recurso (Pre-Flight)

La tabla anterior de métodos y acciones a realizar es una sugerencia, la gran mayoría de APIs implementan las acciones de acuerdo a esta tabla, pero puede existir alguna API que para la alta use `PUT` y para la modificación use `POST`.

Adicionalmente un cliente podría no soportar todos los métodos, por ejemplo un navegador solo soporta `GET` y `POST` en el código HTML. Aunque por medio de Javascript puede soportar todos los métodos.

API REST (Definiciones)

Metadatos

Los metadatos son la información adicional que se transmite junto co el recurso.
Por ejemplo su longitud y tipo de contenido se pueden incluir en la respuesta como encabezados HTTP.
Otra opción es incluir campos adicionales con la respuesta, como el total de registros incluidos.
O enviar una respuesta HATEOAS que son enlaces en la respuesta hacia otras acciones.

Respuesta con campos adicionales:

```
{ "estudiantes": { "id": 10,
                   "name": "Juan",
                   "apellidop": "Perez",
                 },
  "registros": 1  }
```

Respuesta HATEOAS

```
{ "estudiantes": { "id": 10,
                   "name": "Juan",
                   "apellidop": "Perez",
                   "apellidom": "Gonzalez"
                 },
  "links": [{ "rel": "horario",
              "href": "/estudiantes/10/horario"},
            { "rel": "reticula",
              "href": "/estudiantes/10/reticula"}
           ]
}
```

API REST (Definiciones)

Representaciones

Son las diferentes maneras que tenemos de representar un recurso. Un recurso puede tener muchas representaciones.
Por ejemplo, el registro de un empleado en una base de datos lo podemos representar en Json, en Xml, en Texto con Pipes.

```
{
    "id" :50,
    "nombre":"Carlos",
    "apellidos":"Zamora",
    "sueldo":2000,
    "fechaIngreso":"19/05/2010"
}
```

```
<empleado>
    <id>50</id>
    <nombre>Carlos<nombre>
    <apellidos>Zamora<apellidos>
    <sueldo>2000<sueldo>
    <fechaIngreso>19/05/2010<fechaIngreso>
</empleado>
```

```
50|Carlos|Zamora|200019/05/2010
```

API REST (Definiciones)

El cliente puede indicar que representación quiere del recurso

Una Api puede devolver varias representaciones de un mismo recurso, pero el cliente deberá indicar en la petición la representación que desea:

Mediante encabezados de negociación de contenido

El cliente deberá enviar en la petición el encabezado `Accept` indicando los tipos de contenido que puede leer.
`Accept: <MIME-type>/<MIME-subtype>` pudiendo especificar un factor de prioridad con el sub fijo `:q=`. Ejemplo:

```
Accept: text/html, text/plain; q=0.9, text/* ;q=0.8, */* ;q=0.7
```

El servidor deberá leer el encabezado `Accept` y seleccionar una representación para el recurso solicitado de acuerdo a las que soporte el cliente.

Mediante extensiones de archivo

Es una alternativa mas simple que a veces se prefiere sobre la negociación de contenido, ya que una extensión de archivo le indica siempre al programa que lo trate de abrir, el tipo de contenido que tiene.
De manera que si agregamos la extensión a la URL el servidor podrá darse cuenta la representación que se solicita:

```
GET /empresa/empleados.json
GET /empresa/empleados.xml
```

API REST (Definiciones)

Transferencia del Estado Representacional

Estado Representacional

Este término se refiere a el estado de un recurso (y los recursos tienen representaciones).
Como mencionábamos anteriormente un recurso que tengamos almacenado en la base de datos puede ser cualquier cosa, dependiendo de nuestra app podrían ser empleados, una lista de tareas, información de vuelos boletos para un evento, cualquier cosa puede ser un recurso.
Si estuviéramos hablando de una app de reserva de vuelos, la información del vuelo sería el recurso que vamos a almacenar. El estado del recursos sería si el vuelo ya llegó, si esta abordando, si trae retraso, si esta en espera en pista, etc.

Transferencia

En una API REST estaremos transfiriendo el estado que representa un recurso del servidor (back-end/server) a el cliente (front-end/App-Navegador)

- El front-end es la capa de presentación e interacción (la app).
- El back-end es el servidor donde se almacenan los recursos.
- El API es la parte de código que conecta el servidor con la app.

API REST (Definiciones)